

5교시: 이미지 빌드 문제 해결

수업 목표

- build context, .dockerignore, cache, path 오류, 권한 오류를 분류한다.
- 실패 메시지를 복사하지 않고 핵심 증상과 원인을 기록한다.
- docker build --no-cache, docker history, docker inspect를 언제 사용할지 판단한다.

50분 흐름

시간	활동	비중	학생 산출
0-8분	build 실패 유형 소개	설명 15%	failure map
8-25분	context/path 오류 drill	실행 35%	error evidence
25-35분	cache와 .dockerignore 확인	실행 20%	cache note
35-45분	history/inspect 확인	실행 20%	layer evidence
45-50분	RCA mini note 작성	설명 10%	RCA note

Visual 1: Docker build troubleshooting

Week 2 Day 2
Lesson 5

빌드 문제 해결

기본 원칙
증거로 원인을 찾고, 한 가지씩 수정 후 재검증!

빌드 흐름 이해

- 1 Build Context (빌드 컨텍스트)**
docker build <PATH>
지정된 경로의 파일이 모두 전송됨
- 2 .dockerignore**
불필요한 파일/폴더를 제외하여 전송량과 캐시 오염을 줄임
- 3 Dockerfile 실행**
COPY, RUN 등 명령 순서대로 이미지 레이어 생성
- 4 Cache (레이어 캐시)**
변경이 없으면 기존 레이어 재사용
빠르고 안정적인 빌드 가능

지역 단서 확인

- ☒ Dockerfile 경로/문법 확인
- ☒ Build Context 경로 확인
- ☒ .dockerignore 설정 확인

증상(문제 유형)

경로 오류
파일을 찾을 수 없음
(file not found)

Build Context 과다 전송
빌드 느림, 용량 큼

Cache 혼동
변경했는데 반영되지 않음

권한 오류
permission denied

COPY 실패
특정 COPY에서 실패

증거(Evidence)

```
> docker build -t myapp .
...
COPY failed: file not found in build context or excluded by .dockerignore
```

```
> docker build -t myapp .
Sending build context to Docker daemon 1.23GB
...
```

```
> docker build -t myapp .
[cached] RUN npm install
[cached] COPY ..
```

```
> docker build -t myapp .
COPY failed: stat /app/script.sh: permission denied
```

```
> docker build -t myapp .
COPY failed: file not found
...
```

원인 & 해결(Fix)

원인
COPY 경로가 Build Context 기준이 아님 또는 .dockerignore로 제외됨
해결
✓ Build Context와 COPY 경로 다시 확인
✓ .dockerignore에서 제외된 항목 제거

원인
불필요한 파일들이 Context에 포함됨
해결
.dockerignore에 불필요 파일/폴더 추가 (node_modules/, .git/, *.log 등)

원인
캐시된 레이어가 재사용됨
해결
✓ Dockerfile 순서 확인 (변경 파일 먼저 COPY)
✓ 캐시 무시하고 재빌드 (--no-cache)

원인
호스트 파일 권한 문제
해결
✓ 파일 권한 확인 후 수정
✓ chmod +x script.sh

원인
파일/폴더 없음 또는 경로 오타
해결
파일 존재 여부 확인 및 경로 수정

명령어 도구

기본 빌드

```
> docker build -t myapp .
```

 Build Context 기준으로 빌드

캐시 무시 재빌드

```
> docker build --no-cache -t myapp .
```

 모든 레이어를 새로 빌드

이미지 히스토리 확인

```
> docker history myapp
```

 레이어와 크기, 명령 확인

이미지 목록

```
> docker images
```

 빌드된 이미지 확인

빌드 출력 자세히

```
> docker build --progress=plain -t myapp .
```

 상세 로그로 원인 파악

재빌드 순서 (권장 루틴)

증상 확인

증거 수집

원인 추정

수정 반영

재빌드

결과 확인

증거 기록 (Build Log Book)

날짜/시간: _____
문제 유형: _____
명령어: _____

증상(에러 메시지): _____
원인(가설): _____
수정 내용: _____
결과: ☐ 해결됨 ☐ 미해결 (추가 조사)

스크린샷/로그 파일: _____

체크 포인트

☐ Build Context 최소화 (.dockerignore)

☐ COPY 경로는 Context 기준

☐ 캐시 여부 확인 및 --no-cache 재빌드

☐ 권한/파일 존재 여부 확인

★ **기억하세요!**

문제를 발견하면 즉시 증거를 기록하고 공유하세요.

한 번에 한 가지씩 수정하고 재검증하는 것이 빠른 해결의 지름길입니다.

정확한 빌드 = 안정적인 배포의 시작

file:///tmp/lecture-pdf-vmcj9lp6/index.html

1/6

이 이미지는 build 실패를 증상, evidence, 조치로 나누어 본다. COPY 실패는 대부분 build context와 path를 먼저 확인한다.

문제 해결 명령

```
docker build -t paperclip-static-site:day2 .
docker build --no-cache -t paperclip-static-site:day2-nocache .
docker history paperclip-static-site:day2
docker inspect paperclip-static-site:day2
```

전체 failure drill은 hands-on-lab.md의 Phase E를 따른다. 수업에서는 최소 하나의 failure drill을 직접 수행하고, 빠른 학생은 port 충돌과 cache 의심 drill까지 수행한다.

Linux 사전 테스트 결과

docker history paperclip-static-site:day2 핵심 출력:

IMAGE	CREATED	CREATED BY	SIZE
2769096ea8e1	...	EXPOSE [80/tcp]	0B
<missing>	...	COPY index.html styles.css ./	2.34kB
<missing>	...	WORKDIR /usr/share/nginx/html	0B

docker inspect 핵심 확인:

```
"WorkingDir": "/usr/share/nginx/html"
"Cmd": ["nginx", "-g", "daemon off;"]
"ExposedPorts": {"80/tcp": {}}
"Size": 48241686
```

build fault taxonomy

Dockerfile 오류는 단순 syntax 문제가 아니라 build model을 잘못 이해해서 생기는 경우가 많다. 다음 네 가지 축으로 먼저 나눈다.

축	질문	대표 evidence
Context	Docker daemon에 어떤 파일이 전달됐는가	transferring context, .dockerignore
Path	COPY 기준 경로가 context 안에 있는가	COPY failed, ls
Cache	이전 결과가 재사용됐는가	CACHED, --no-cache
Runtime confusion	build 문제가 아니라 실행 문제인가	build 성공 후 docker run, logs

학술적으로는 Dockerfile을 build artifact를 만드는 source code로 볼 수 있다. Dockerfile fault 연구들이 instruction 실행 전후의 context, path, layer 변화를 분석 대상으로 삼는 이유도 여기에 있다. 학생은 실패 메시지를 외우는 것이 아니라 "어느 build input 또는 build step이 잘못되었는가"로 좁혀야 한다.

cache drill

styles.css의 색상 하나만 바꾸고 다시 build하면 보통 FROM, WORKDIR 등 앞 step은 재사용되고 COPY 이후가 다시 처리된다. cache는 시간을 줄이지만, 예상과 다르게 오래된 파일을 보고 있다고 의심되면 --no-cache로 한 번 검증한다.

```
docker build --no-cache -t paperclip-static-site:day2-nocache .
```

--no-cache는 매번 쓰는 옵션이 아니다. 원인 확인용으로 사용하고, 평소에는 Dockerfile instruction 순서와 COPY 범위를 조정해 cache가 자연스럽게 효율적으로 작동하게 한다.

failure table

증상	먼저 볼 것	조치
COPY failed	현재 directory, build context	pwd, ls, Dockerfile path 확인
context가 너무 큼	build output context size	.dockerignore 보완
수정했는데 반영 안 됨	cache 사용 여부	필요한 경우 --no-cache
permission denied	file permission, Docker socket	권한/소유자/daemon 상태 확인
base image pull 실패	image name, network	tag/registry/network 확인

RCA mini note

Build Failure Note

- 명령:
- 실패 메시지 핵심:
- 정상 기준:
- 의심 원인:
- 확인한 evidence:
- 조치:
- 재확인 결과:

추가 failure drill 선택지

Drill	일부러 만드는 문제	학습 목표
bad COPY	context 밖 파일 복사 시도	build context boundary
wrong port	curl을 다른 host port로 실행	host/container port 구분
stale image	source 수정 후 rebuild 생략	image build와 bind mount 차이
no-cache	cache 의심 상황 재빌드	cache 검증 도구

핵심 유의사항

5교시는 일부러 실패를 다루지만, 실제 파일을 무작정 망가뜨리게 하면 복구 시간이 길어진다. 가능한 경우 별도 Dockerfile 이름이나 별도 tag를 사용한다. 예를 들어 Dockerfile.bad-copy를 읽는 상황을 설명하거나, tag를 day2-nocache, day2-debug처럼 분리한다.

실패 메시지를 통째로 복사하는 것은 좋은 evidence가 아니다. 실패 메시지의 첫 원인 줄, 실행한 명령, 현재 directory, 기대한 정상 기준을 함께 기록한다. 장애 분석은 "에러가 났다"가 아니라 "어떤 입력과 어떤 단계에서 정상 기준을 벗어났다"로 정리해야 한다.

권한 오류는 두 종류로 나뉘야 한다. 하나는 Docker daemon socket 접근 권한이고, 다른 하나는 build context 안 파일 권한이다. 전자는 docker ps도 실패하는 경우가 많고, 후자는 특정 파일을 COPY하거나 읽는 단계에서 실패한다. 둘을 섞으면 엉뚱한 조치를 하게 된다.

오류별 첫 질문

오류 유형	첫 질문	다음 evidence
context/path	지금 어느 directory에서 build했는가	pwd, ls -la
cache	정말 새 source로 build했는가	build output의 CACHED, image tag
permission	Docker 자체가 안 되는가, 파일만 안 되는가	docker ps, file permission
registry/network	base image를 가져오지 못했는가	image name, tag, network
runtime confusion	build는 성공했는데 실행이 실패했는가	docker run, docker logs

좋은 RCA와 나쁜 RCA

나쁜 기록	좋은 기록
Docker가 안 됨	docker build의 COPY index.html step에서 파일을 찾지 못함
cache 문제 같음	source 수정 후 docker build output에서 COPY step이 CACHED였음
port가 이상함	-p 18082:80로 실행했지만 curl localhost:80을 호출함
이미지가 안 뜸	tag day2-edit로 build했지만 run은 day2를 사용함
권한 문제	docker ps부터 socket permission denied가 발생함

50분 진행 흐름

0~10분에는 failure taxonomy를 먼저 본다. 에러 메시지를 보자마자 검색하기보다 context, path, cache, permission, runtime 중 하나로 먼저 분류한다.

10~30분에는 최소 하나의 drill을 수행한다. 기본 drill을 완료한 뒤에는 port 충돌이나 stale image drill로 확장한다.

30~45분에는 docker history와 docker inspect를 읽는다. JSON 전체를 해석하려고 하기보다 WorkingDir, Cmd, ExposedPorts, Size처럼 오늘 필요한 키만 찾는다.

45~50분에는 RCA mini note를 완성한다. 실패를 일부러 만들었더라도 마지막 줄은 반드시 재확인 결과여야 한다. "고쳤다"가 아니라 어떤 명령으로 정상 확인했는지 적는다.

학생 기록 템플릿 확장

```
## Lesson 5 Troubleshooting Evidence
```

- 실패 유형:
- 실행한 명령:
- 현재 directory:
- 실패 step:
- 핵심 error line:
- 정상이라면 보여야 할 output:
- 내가 확인한 evidence:
- 조치:
- 재검증 명령:
- 재검증 결과:

평가 기준

기준	2점 evidence
분류	context/cache/path/permission 중 어느 문제인지 분류했다.
명령	history 또는 inspect로 image 상태를 확인했다.
보안	secret/log/screenshot이 context에 들어가지 않게 설명했다.
RCA	실패와 재확인 결과를 기록했다.

공식 근거 링크

- Docker build context: <https://docs.docker.com/build/concepts/context/>
- Docker image history: <https://docs.docker.com/reference/cli/docker/image/history/>
- Dockerfile reference: <https://docs.docker.com/reference/dockerfile/>